

# melonYX Engine Phase 1: Particle System

Implementation Document

**Engine name:** melonYX

**Author:** Erica Mauriz Barundia

**Last Updated:** June 05, 2026

---

## Overview

The melonYX engine is a lightweight particle physics engine built in C++ using OpenGL for rendering. Phase 1 implements a **fountain firework particle system** where sparks are continuously spawned, simulated under gravity, and rendered as 3D spheres.

## Libraries Used

- **OpenGL** - GPU rendering pipeline
- **GLFW** - window creation and input handling
- **Glad** - OpenGL function loader
- **GLM** - math library for vectors and matrices
- **tinyobjloader** - loading the sphere.obj 3D model

## Particle Class

Each particle is an instance of the Particle class stored inside the engine.

```
C/C++
class Particle {
public:
    glm::vec3 position;
    glm::vec3 velocity;
    glm::vec3 acceleration;
    glm::vec3 color;
    float lifespan;
    float radius;
};
```

It contains:

- **position** - current world-space location (glm::vec3)

- **velocity** - movement per second (glm::vec3)
- **acceleration** - set to (0, -10, 0) for standard gravity at 10 m/s<sup>2</sup>
- **color** - randomly assigned RGB value
- **lifespan** - randomly assigned between 1 and 10 seconds; decremented each frame
- **radius** - randomly assigned between 2 and 10 meters; used to scale the sphere mesh

When a particle's lifespan reaches zero, it is removed from the engine's particle list and **no longer** updated or rendered.

---

## Engine Update Loop

The `melonyx::update(float dt)` function is called **once** per frame with a fixed timestep of 1/60 seconds. It performs two tasks:

1. **Spawning** - `continuousSpawn(dt)` accumulates elapsed time and spawns batches of particles at a fixed interval. The number of particles per batch is derived from the user-defined spark count entered at launch.
2. **Simulation** - Each active particle is updated using Euler integration:
  - `velocity += acceleration * dt`
  - `position += velocity * dt`

Particles whose lifespan has expired are erased from the list.

---

## Spawning

Particles spawn at a fixed elevated point near the bottom of the 800m world space at position (0, -280, 0). Each spark is given a random upward velocity with a wide horizontal spread to produce the fountain effect. Gravity pulls them back down over their lifespan.

```
C/C++
for (int i = 0; i < spawnCount; ++i) {
    glm::vec3 vel(
        (rand() % 200 - 100) * 0.5f,    ///< X: -50 to +50 m/s
        (rand() % 100 + 50) * 0.8f,    ///< Y: 40 to 120 m/s upward
        (rand() % 200 - 100) * 0.5f    ///< Z: -50 to +50 m/s
    );
    ...
};
```

## Rendering

The sphere mesh is loaded from 3D/sphere.obj using tinyobjloader and uploaded to the GPU once at startup. Each frame, for every active particle, a model matrix is computed by translating to the particle's position and scaling by its radius. The model, view, and projection matrices are passed to the vertex shader as uniforms. The fragment shader uses a `particleColor` uniform to apply each particle's individual color.

The camera supports both orthographic and perspective projection, switchable at runtime via the 1 and 2 keys. WASD rotates the camera orbitally around the world origin. Space toggles pause.

---

## Controls

| Key   | Action                  |
|-------|-------------------------|
| 1     | Orthographic view       |
| 2     | Perspective view        |
| W/S   | Orbit camera up/down    |
| A/D   | Orbit camera left/right |
| Space | Pause / Resume          |